COMP5329 / COMP4329

Deep Learning

Assignment 1: Debugging and Experiments

Inception in QANet: Redesigning QANet's Encoder Block

Group Members

#	Full Name	Student ID	UniKey
1	Nicholas Davies	530483829	ndav4040
2	Nicholas Usich	560361894	nusi0337
3			

Google Drive (Editor) Link	https://drive.google.com/drive/folders/1Ynac3xCG86hqxWJS8RMHUCOlxfmFzLBc?usp=share_link
Submission Date	*13 April 2026*

Introduction

This report investigates the role of the encoder block during training and the effect of incorporating an inception based encoder block within the University of Sydney COMP4329 2026 implementation of QANet. Replacing the encoder's convolutional layers with an inception convolutional model provides an architectural change through which the deep learning mechanisms behind the model can be properly explored.

This report will explore three hypotheses that focus on the effects of changing to the inception based module. Specifically these hypotheses can be defined as:

Hypothesis 1: The inception based model will increase performance through F1 and EM metrics on the testing set as it is better at capturing multi-scale contextual interactions. The model will accelerate convergence resulting in a lower loss at each epoch relative to the baseline. The Inception based model will have an increased cost of computational power and parameter count when compared to the base model.

Hypothesis 2: Inverting the encoder block to process global context (attention layers) before local context (inception based convolutional layers) will reduce F1 and EM scores and result in a higher final training loss compared to the baseline, as local feature extraction is more effective when applied to raw representations before global context is introduced.

Hypothesis 3: Adding group normalisation to the Inception model will reduce training loss variance across steps and improve generalisation, compared to the same architecture using layer normalisation.

Before these investigations can be conducted, the initial codebase implementation is flawed, containing both runtime and mechanism issues that must be fixed. The patches, methodology and solutions for these problems are outlined within stages I and II, whilst the experiment can be found within stage III.

Fundamental pipeline (Stage I)

This section deals with all bugs that appeared during the runtime execution of the framework. The investigative process is simple, comprised of running the training script, reading the debugging log when a runtime execution occurs, research into what is meant to occur in the affect section of code and then implementation. Below is listed all occurring issues within the runtime execution of the QANet implementation.

Solution Summary Table

For convenience, related bugs have been condensed into the same issue. Detailed descriptions of all issues can be found after the table.

Issue Number	File Location	Issue	Solution
1	train.py	Incorrect amount of arguments given to namespace function	Unpack argument in namespace with **. To get in form a=1, etc.
2	encoder.py	Tensor size mismatch	Change unsqueeze dimension on length (1st) dimension.
3	train.py	Unknown scheduler	Change to known scheduler

4	cosine_scheduler.py	Incorrect function in math library	Change PI to pi.
5	qanet.py	Index out of range caused by wrong dimension	Change so that it will create a word embedding of Cwid and character embedding of Ccid.
6	embedding.py	Dimension Issue	Change so that it swaps to the correct specified dimensions of [B,d_char,L,char_len]
7	conv.py	Size mismatch in padding	Change so that it accounts for dynamic height change.
8	embedding.py	Incompatible Shapes for a matrix multiplicaiton	Transpose on Length and Channels rather than Batch and Channels.
9	conv.py	Dimension Issue	Unfold on the output size.
10	conv.py	Wrong order of operations.	Invert so that a depthwise convolution occurs before a pointwise convolution
11	layernom.py	Dimension Mismatch	Have mean and variance keep their input dimensions for output.
12	encoder.py	Iteration index out of range	Change so that iteration matches self.convs. I+1 to i.
13	qanet.py	Tensor Size Mismatch	In function call swap cmask and qmask arguments to match parameters.
14	attention.py	Dimension issue for bmm.	Swap S1 and Q for A and have S1 bmm with a bmm between S and C.
15	heads.py	Incompatible Shapes for a matrix multiplicaiton	Concatenate on the channel dimension to get a shape of [B,2C,L]
16	loss.py	Multi-Target Tensor given.	Swap p1 and y1 to that y1 is the target and p1 is the input to nll_loss
17	train_utils.py	Incompatible type for calling backward	Remove item() so that .backward() can do backpropagation on loss.

18	evaluate.py	Key issue when loading ‘model’	Change model to valid key model_state
19	sgd_momentum.py	Key issue is velocity doesnt exist in state	Change “vel” to “velocity”
20	adam.py	Key issue when loakding “m” and “v”	Change “m” to “exp_avg” and “v” to “ exp_avg_sq”
21	evaluate.py	Pickle Data load Failure	Set weights_only to false.
22	evaluate.py	Issue with reconstructing model with args	Unable to replicate

Detailed Bug Description

Issue No: 1
Error: Error in arguments when creating a namespace, specifically copying the locals dictionary. This causes an error in “train.py”.

```
TypeError: Namespace.__init__() takes 1 positional argument but 2 were given
```

Solution: Unpack using ** the arguments so that they are in the form of keyword1 = a, keyword2 = b etc.

```
args_dict = {k: v for k, v in locals().items()}
args = argparse.Namespace(**args_dict)
```

Issue No: 2
Error: Tensor size mismatch in the between pos and freqs matrix in “encoder.py”. Freqs is being unpacked into size [C,1] or [96,1], unsqueeze(0) turns it into [1,C].

```
RuntimeError: The size of tensor a (400) must match the size of tensor b (96) at non-singleton dimension 1
```

Solution: To multiply tensors we need [C,1] and [C,L]. When defining freqs, unsqueeze on the 1st not the 0 dimension.

```
freqs = torch.tensor(
    [10000 ** (-i / d_model) if i % 2 == 0 else -10000 ** ((1 - i) / d_model) for i in range(d_model)],
    dtype=torch.float32
).unsqueeze(1)  # [C, 1]
```

Issue No: 3
Error: none is not a valid scheduler passed into the train function defined in train.py. This causes an error in “assignment1.ipynb”.

```
ValueError: Unknown scheduler 'none'. Available: ['cosine', 'step', 'lambda']
```

Solution:
Change none to one of the three available schedulers, for example “cosine”. This will allow for a valid scheduler to apply to the learning rate.

Issue No: 4

Error: Incorrect function call to the math library in cosine_scheduler.py

```
AttributeError: module 'math' has no attribute 'PI'
```

Solution: the correct function call to get pi is math.pi

```
self.eta_min + (base_lr - self.eta_min) * (1 + math.cos(math.pi * t / self.T_max))
```

Issue No: 5
Error: Index out of range caused by wrong embeddings in “qanet.py”.

```
-> 2567 return torch.embedding(weight, input, padding_idx, scale_grad_by_freq, sparse)

IndexError: index out of range in self
```

Solution: The Cw is accidentally a character embedding and Cchar is a word embedding. Swap word and character embeddings so that Cw is a word embedding of Cwid and Cc is the character embedding of Ccid.

```
Cw, Cc = self.word_emb(Cwid), self.char_emb(Ccid)
```

Issue No: 6
Error: Incorrect shape of tensor for a view operation in “embedding.py“.

```
RuntimeError: shape '[1, 64, 16, 1, 1]' is invalid for input of size 4096
```

Solution: Using permute swap the 3 and 2 dimensions, as specified in the commented shapes so that ch_emb = [B,d_char, L,char_len].

```
ch_emb = ch_emb.permute(0, 3, 1, 2)  # [B, d_char, L, char_len]
```

Issue No: 7
Error: Size mismatch in “conv.py”, this is due to incorrect padding of the width for x.

```
RuntimeError: Sizes of tensors must match except in dimension 3. Expected size 400 but got size 404 for tensor number 1 in the list.
```

Solution: When the height is padded previously, it does not account for this in when padding on the width, this leads to a height mismatch. So pad for entire height of the new height.

```
pad_w = x.new_zeros(B, C_in, x.size(2), p)
```

Issue No: 8
Error: Matrix size mismatch for multiplication in “embedding.py” .

```
RuntimeError: mat1 and mat2 shapes cannot be multiplied (145600x8 and 364x364)
```

Solution: The matrix x is transposed swapping the 0 and 2nd dimension. It should instead be transposed on the 1st and 2nd dimension.

```
# x: [B, C, L] -> [B, L, C]
x = x.transpose(1, 2)
```

Issue No: 9
Error: Incompatible shape for view operation in “conv.py”.

```
RuntimeError: shape '[8, 364, 0, 400, 5]' is invalid for input of size 1486720
```

Solution: It is currently unfolding x on the C_in dimension, it should be sliding on the L_out to get the desired amount of windows.

```
x_unf = x.unfold(2, self.kernel_size, 1)  # [B, C_in, L_out, k]
```

Issue No: 10

Error: Incompatible shape for view operation in “conv.py”.

```
RuntimeError: shape '[8, 364, 0, 400, 5]' is invalid for input of size 1536000
```

Solution: When depthwisesperableconv is run, it first does a pointwise before running a depthwise. This should be reversed so that depthwise runs first, this will allow for a correct shape.

```
def forward(self, x):
    return self.pointwise_conv(self.depthwise_conv(x))
```

Issue No: 11
Error: Tensor mismatch in size within “layernorm.py”

```
RuntimeError: The size of tensor a (400) must match the size of tensor b (8) at non-singleton dimension 2
```

Solution: Caused by the dimensions not being kept within the mean and variance. To fix change both mean and var to keep their input dimensions.

```
mean = x.mean(dim=dims, keepdim=True)
var = x.var(dim=dims, keepdim=True, unbiased=False)
```

Issue No: 12
Error: Iteration out of index in “encoder.py”

```
IndexError: index 4 is out of range
```

Solution: As it goes through self.convs, the i + 1 will be out of range for the self.norms indexing as both are defined based on the same size conv_num. Therefore change i+1 to i.

```
out = self.norms[i](out)
```

Issue No: 13
Error: Tensor mismatch in size within “attention.py”, caused by the function call within “qanet.py”

```
RuntimeError: The size of tensor a (400) must match the size of tensor b (50) at non-singleton dimension 2
```

Solution: when cq_att is called, the cmask and qmask are passed in the wrong places. Swapping with cmask and qmask in call with fix it.

```
X = self.cq_att(Ce, Qe, cmask, qmask)
```

Issue No: 14
Error: Issue with bmm multiplication in “attention.py”

```
RuntimeError: Expected size for first two dimensions of batch2 tensor to be: [8, 96] but got: [8, 400].
```

Bmm does a matrix multiplication, if you swap q and s1 it will have correct dimensions. If you change the second line to have a matrix multiplicaiton between s1, and then C, you will get the correct training.

```
A = torch.bmm(S1, Q)
B = torch.bmm(S1, torch.bmm(S2.transpose(1, 2), C))
```

Issue No: 15
Error: Size mismatch of matrices in matrix multiplication operation in “heads.py”

```
RuntimeError: size mismatch, got input (6400), mat (6400x96), vec (192)
```

Solution: As the X1 matrix is concatenated on the 0 dimension it adds on the batch not the channel dimension. To fix concatenate on the channel dimension 1. This will produce the correct shape of 2C.

```
X1 = torch.cat([M1, M2], dim=1)  # [B, 2C, L]
```

Issue No: 16
Error: Multiple targets given in nll_loss function call in “loss.py”

```
RuntimeError: 0D or 1D target tensor expected, multi-target not supported
```

Solution: The nll_loss function needs an input and then a target. The function call F.null_loss(y1,p1) gives a 2 dimensional target for p1. To fix, swapping y1 and p1 will produce the correct target and input tensors.

```
return 0.5 * (F.nll_loss(p1, y1) + F.nll_loss(p2, y2))
```

Issue No: 17
Error: Incorrect type for function call in “train_utils.py”.

```
AttributeError: 'float' object has no attribute 'backward'
```

Solution: in PyTorch the .item() method is meant to convert a single element tensor to a float or integer. Therefore backwards cannot be called on a float, and so .item() must be removed, to perform the backpropagation.

```
loss.backward()
```

Issue No: 18
Error: In “Evaluate.py” model is not a valid key to extract from the checkpoint.

```
KeyError: 'model'
```

Solution: As model does not exist as a key, load state of model from model_state.

```
model.load_state_dict(ckpt["model_state"])
```

Issue No: 19
Error: In “SGD_Momentum.py”, the program tries to assign to state[“vel”] which does not exist.
Solution: As “velocity” does not exist as a key, set “velocity” to zeros.

```
state["velocity"] = torch.zeros_like(p)
```

Issue No: 20
Error: adam tries to access m and v as keys from state
Solution: Following the zero values above, grab m from “exp_avg” and v from “exp_avg_sq”.

```
m, v = state["exp_avg"], state["exp_avg_sq"]
```

Issue No: 21
Error: Trying to load with weights_only will be false and will crash.

Solution: As the model is saved with more than just weights, set it to False so that it can correctly load.

```
ckpt_path = os.path.join(save_dir, ckpt_name)
ckpt = torch.load(ckpt_path, map_location=DEVICE, weights_only=False)
```

Issue No: 22
Error: Args didn't contain parameters necessary for reconstructing the proper model.
Solution: To fix this use the arguments provided within the checkpoint loaded with pickle.

```
args = argparse.Namespace(**ckpt["config"])
word_mat, char_mat = load_word_char_mats(args)
```

After the solutions to all of the above bugs were implemented, a training and loading iteration could be conducted.

Deep Learning Mechanisms (Stage II)

The mechanisms of qanet were then investigated through two methods. The first was to go through each module used within the code to ensure that the correct formulas were being implemented. The second was then to observe training loss and metrics to ensure proper training.

Mechanism Implementation Investigation

This section of the report covers any adaptation of the initial scaffold to ensure that the implemented mechanisms were correct. Each change will be displayed with a description and an image of the change, with the original implementation displayed on the left (red) and the updated on the right (green).

These changes can be further broken down into the following categories.

Weight Initialisation

The scaffold includes two types of weight initialisation schemes, kaiming and xavier, and for both there are also implementations to generate with two types of distribution, normal and uniform distributions. These weights are used to initialise the weights for the convolutional layer in the DepthwiseSeperableConv function.

Within Kaiming, for both distributions the equation used for standard deviation is incorrect. The implementation uses 1.0 divided by fan when it should be 2.0.

```
std = math.sqrt(1.0 / fan)        25+    std = math.sqrt(2.0 / fan)
```

Within the Xavier implementations, for both distributions the calculations for standard deviation are incorrect, multiplying instead of adding fan in and fan out.

```
std = gain * math.sqrt(2.0 / (fan_in * fan_out))   →   24+    std = gain * math.sqrt(2.0 / (fan_in + fan_out))
```

Standard deviation was calculated using multiplication instead of addition in the denominator (fan_in * fan_out) instead of (fan_in + fan_out). This would cause a very small standard deviation, and after a few layers its activation would be near-zero. The signals would be almost zero, causing vanishing gradients and a model that cannot learn. If the standard deviation of the weights was calculated with a 1 in the numerator instead of a 2, the signals would shrink because ReLU removes half the signal when it changes all negative numbers to zero (GeeksforGeeks, 2025). Ultimately, this would cause a vanishing gradient and a model that cannot learn.

Activation Functions

For activation functions, there are two implementations available, ReLU and it's extension Leaky ReLU. These are used in both the embedding and encoder blocks.

The ReLU function was incorrectly implemented, inverting the minimum floor of x and instead having a maximum cap of x at 0.

The leaky ReLU function was incorrectly implemented by having values of x less than 0 be set to x whilst having all values greater than x being multiplied by the negative slope. This should be reversed with all values less than 0 being multiplied by the slope.

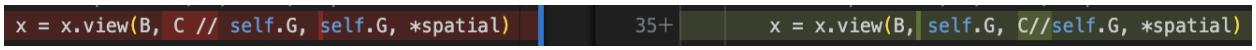
Without these fixes, activations are either solely negative or have suppressed positive values. This would impair the signal distribution and reduce gradient flow, hindering optimization. Training would be ineffective and unstable.

Normalisation Functions

There are two normalisation functions that are implemented in the base code, group norm and layer norm. These are used in the Encoder Block.

Within layernorm, the application of weights and bias were not correctly applied. Originally multiplying the x_norm by the bias and adding the weights. This was adjusted by correctly multiplying by the weights and adding the bias.

Within GroupNorm, in the original codebase when reshaping the tensor into the groups, it is reshaped to the wrong shape. Swapping the channel and grouped dimensions.



There was a view operation that mixed the group and channel dimension. Information was cross contaminated between channels. Normalization would occur between a mix of channels and groups. This would result in poor normalization and misaligned values between layers. The mean and variance are calculated using the wrong numbers, and channels that are supposed to be independent are no longer independent, destabilizing training. In LayerNorm, bias is initialized to zeros, and weights are initialized to ones. In this incorrect version of LayerNorm, it multiplies by bias which scales everything to zero, and then adds weights which are 1. This results in a constant output that eliminates variance and prevents learning. (Xu et al, 2019)

Dropout Component

Within the dropout implementation used within Embedding, attention and encoder block. The scaling was not implemented correctly, diving all values by p instead of 1-p.

Without this fix, activations would either explode or vanish, because they are being scaled by the raw probability. Each activation should be scaled by 1/(1-p) to preserve the expected value of all activations post-dropout. If implemented with 1/p, the value of activations would be far too high.

Optimiser Modules

There are three different optimisers that are implemented within the codebase, sgd, sgd momentum and adam. These are used within training.

For both SGD and Adam optimisers as the weight decay value is a positive value, the calculations of the gradient are incorrect. If weight decay (or l2 regularisation in this case) is used, when calculating the derivative (gradient), the decay applied to each weight is implemented with the wrong sign. This causes weights to increase instead of decay, leading to exploding gradients. For all optimisers the weight decay will remain positive in the gradient calculation.

```
grad = grad.add(p, alpha=-wd)
```

```
39+ grad = grad.add(p, alpha=wd)
```

Within SGD with momentum, there is a calculation error in that the gradient was subtracted from the velocity rather than added.

```
v.mul_(mu).sub_(grad)
```

```
55+ v.mul_(mu).add_(grad)
```

However, the update rule specified in the provided formula is inconsistent with Sutskever et al. implementation, in this case we are implementing the PyTorch model. The code applying the learning rate to not just the gradient but also the momentum and velocity term. (PyTorch Contributors, n.d. -c)

If we were to implement the course note model. We could do this by removing lr from the p calculation and applying it directly to the gradient and then subtracting v from p , this new implementation is in line with the course notes.

```
# v = momentum * v + grad
v.mul_(mu).sub_(grad)

p.add_(v, alpha=-lr)
```

```
59+ # v = momentum * v + (grad * lr)
60+ v.mul_(mu).add_(grad,alpha=lr)
61+ # p = p - v
62+ p.sub_(v)
```

Within Adam, there are several miscalculations, mistakenly placing multiply instead of exponentiation in the bias correct and velocity estimate stages.

For the velocity estimate stage, the calculations do not square the gradient. And for the bias corrects it does not raise the beta values to the power of t.

```
# Update biased moment estimates
m.mul_(beta1).add_(grad, alpha=1.0 - beta1)
v.mul_(beta2).add_(grad, alpha=1.0 - beta2)

# Bias correction
bias_correction1 = 1.0 - beta1 * t
bias_correction2 = 1.0 - beta2 * t
```

```
66 # Update biased moment estimates
67 m.mul_(beta1).add_(grad, alpha=1.0 - beta1)
68+ v.mul_(beta2).add_((grad**2), alpha=1.0 - beta2)
69
70 # Bias correction
71+ bias_correction1 = 1.0 - (beta1 ** t)
72+ bias_correction2 = 1.0 - (beta2 ** t)
```

Without these updates, the optimizers would not produce properly scaled, correctly directed steps. In SGD and Adam, the l2 regularization was subtracted instead of added, which would have increased weights over time, not made them converge to zero. Models aim to have sparse representations with non-meaningful weights at zero, and this would reverse this, causing massive weights, massive numbers, and exploding gradients. With Adam, g was not squared, removing the variance estimate and causing Adam to not be adaptive. (PyTorch Contributors, n.d. -a) In SGD with Momentum, the momentum is subtracted instead of added, which causes oscillation and partial gradient ascent. This makes optimization unstable, reducing training efficacy.

Learning Rate Schedulers

The scaffold contains three scheduler implementations, cosine annealing, lambda and step. These are used within the training function.

Within cosine annealing, the implementation is incorrect as it does not half all of the later terms in the calculation.

```
self.eta_min + (base_lr - self.eta_min) * (1 + math.cos(math.PI * t / self.T_max))
self.eta_min + 0.5 * (base_lr - self.eta_min) * (1 + math.cos(math.pi * t / self.T_max))
```

Within the lambda scheduler implementation, to update each lr it mistakenly adds the factor of change rather than multiplies by it.

```
return [base_lr + factor for base_lr in self.base_lrs]
23+    return [base_lr * factor for base_lr in self.base_lrs]
```

For the step scheduler, the implementation of the power is incorrectly done, using the symbol * instead of **. This will only multiply the terms rather than raise to the power of the secondary symbol.

```
base_lr * self.gamma * (t // self.step_size)
25+    base_lr * (self.gamma ** (t // self.step_size))
```

For all of these, it will result in learning rates that are either too high or too low at certain parts of the training process. A learning rate that is too high in any epoch will overstep out of the optimal descent path, diverging or bouncing around minima. The optimization process will be erratic and unstable. If the learning rate is too low, optimization will take too long, and can get stuck at saddle points. For cosine annealing, not halving will result in a peak learning rate that is too high, and also some learning rates that are too low. For the lambda scheduler, adding the factor of change will consistently increase learning rate, causing large overshooting and ignoring the lambda schedule, potentially diverging. (PyTorch Contributors, n.d. -b).

Loss Functions

Within the training phase there are two implementations of loss functions nll loss and ce loss. The nll loss requires the function to be the output of log_probability, but CE does not (PyTorch Contributors, n.d. -b). However, within qanet, the original implementation had both logits run through softmax regardless of loss function. Hence, we have modified the model function to return p1 and p2 as raw logits and the nll loss function to perform the softmax instead.

```
8+    p1 = F.log_softmax(p1, dim=1)
9+    p2 = F.log_softmax(p2, dim=1)
```

Without this change, cross-entropy loss was receiving softmax probabilities from the model. PyTorch already applies softmax in its cross entropy function, meaning softmax was being applied twice. That flattens the probability distribution, making large differences smaller. It would reduce training signal, as information is lost during the flattening of the second softmax. That causes vanishing gradients and slower convergence.

Encoder Block

Within the implementation of the encoder block, there were several modifications that were included to more align the codebase with the qanet paper.

The position encoding is kept the same as the original implementation.

```
# 1: Pos encoding
out = self.pos(x)
```

All convolutional residuals and layernorm applications are placed directly inside the convolutional for loop. The new convolutional loop is as follows:

```
# 2: convolutional layer
for i, conv in enumerate(self.convs):
    res = out
    out = self.norms[i](out)
    out = conv(out)
    out = self.act(out)
    if (i + 1) % 2 == 0:
        out = self.conv_drops[i](out)
    out = out + res
```

Within the self attention we take a new residual before implementing a layernorm that is not reliant within the convolutional loop. In the original implementation all work by the self attention and layernorm layers is never used as it is immediately replaced by the residual set in the last convolutional loop.

```
out = self.self_att(out, mask)
out = res
out = self.drop(out)
```

In the new implementation, as there is a new residual before both layer norm and attention layers it can properly be concatenated with the output of the attention layer after dropout.

```
# 3: Self attention
res = out
out = self.normb(out)
attn_out = self.self_att(out, mask)
out = res + self.drop(attn_out)
```

For the feedforward stage of the encoder block, the only change implemented was the output of the activation function was put through a dropout layer before being concatenated with the residual.

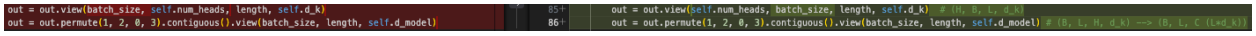
```
# 4: FFN
res = out
out = self.norme(out)
out = self.fc(out.transpose(1, 2)).transpose(1, 2)
out = self.act(out)
out = res + self.drop(out)
return out
```

Within the self attention layer of the encoder block there are also some incorrect calculations. When calculating attention, the scaling factor is forgotten before a softmax is included. (Vaswani et al, 2017)

```
attn = torch.bmm(q, k.transpose(1, 2))
attn = mask_logits(attn, attn_mask)
attn = F.softmax(attn, dim=2)
```

```
80+ attn = self.scale * torch.bmm(q, k.transpose(1, 2))
81+ attn = mask_logits(attn, attn_mask)
82  attn = F.softmax(attn, dim=2)
```

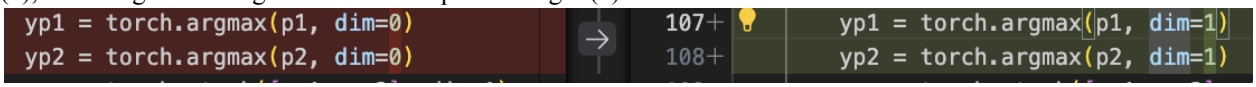
To comply with the permute series in the original codebase, size of the batch_size and num_heads in view needs to change so that in memory the second view can grab batch_size first, length then heads.



Without these fixes, the residual flow bypassed self attention and LayerNorm between layers, the FFN didn't apply dropout, attention was unscaled, and the batch dimension was being swapped with the head dimension. Skipping layering and self attention causes an information flow that is not updated to capture long-range dependencies or have consistent feature scales across layers. Also, they are still being computed so it wastes computational work, for a shallow, partial encoder. Self attention is a critical component for high quality encoding, so removing it would significantly decrease the model's encoding quality, in turn lowering prediction quality (Alammar, 2018). Skipping layer norms would result in inconsistent feature scales, meaning layers would need to spend time and parameters learning how to communicate with their neighboring layers, rather than having norms do it more efficiently (Alammar, Grootendorst, 2024). Using dropout with the FFN is a regularization technique that reduces dependency on any specific neuron, making the model more robust to unseen patterns and reducing overfitting. Without it, prolonged training metrics would not align with testing metrics. Unscaled attention can lead to numerical instability in the softmax, especially as dimensions grow, which would break training. Finally, switching the batch dimension and head dimension cross contaminates information across batches and heads, breaking independence assumptions. Overall, these changes let the encoder correctly propagate, normalize, and transform information into a useful representation for downstream steps.

Evaluation tools

Within the eval_util the argmax was applied to the wrong dimension. Instead of argmaxing over the batch dimension (0), we changed to to argmax over the sequence length (1).



If argmax was carried out on the batch dimension, it would compare values across batches instead of comparing values across words. It would compare different samples to each other, rather than evaluate each sample independently. QANet aims to select two words from the input sequence, so this would result in an output that is not useful for evaluation, as the evaluation works by assessing the two words the model selected, and the two correct words. Overall training would work, but evaluation metrics would be misleading.

Train Tools:

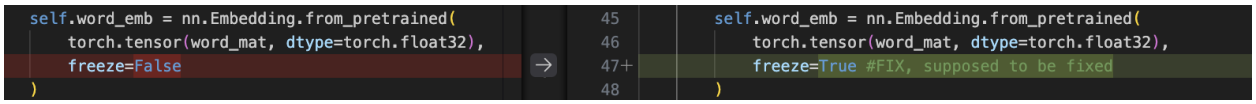
The gradient was getting clipped after the optimizer step, which is the incorrect order. The fix was clipping the gradient before the optimizer steps. The correct order is as follows.

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
optimizer.step()
scheduler.step()
```

Gradient was clipped after the optimizer stepped, which essentially avoids gradient clipping at all. Therefore, exploding gradients have a large effect on optimization making descent unstable and erratic. There was no limit on how large of a step can be taken. This can lead to divergence and poor convergence.

Word Embedding

To better align with the qanet paper, when loading the word embedding from a pretrained tensor, the embedding is meant to be frozen.



These word embeddings are already very high quality and have been reliable for the scientific community. If unfrozen, our model would be more adaptable, but risks overfitting and instability due to overwriting structures and information learned from a large amount of text.

Other modules investigated

Impact of hyperparameter changes were assessed to a varying degrees of success. Most notably, increasing the batch size, which started at 8, to higher values was extremely impactful to convergence. Without the change, the training process would decrease for the first few hundred steps, and plateau with a high loss and poor F1. A small batch size means optimizers attempt to update an entire model (trying to approximate a whole dataset), on just a few samples. Loss was only accumulated over 8 baches, meaning statistics were not significant due to the small batchsize. Gradients were uninformative except for the first few steps, so training plateaued early. The solution was very simple, increasing batchsize to around 16 or 24 had much better results.

In addition to this, the investigation also explored the overall implementation of the qanet architecture defined in its forward function, and the convolutional, embedding and attention modules. These were found to be correctly implemented, without change required.

Training Evaluation (Stage I and II)

To test if the adaptations made within stage I and stage II have produced a working version of the codebase so that experiments can be conducted properly, there are aspects of the model in training that we can explore.

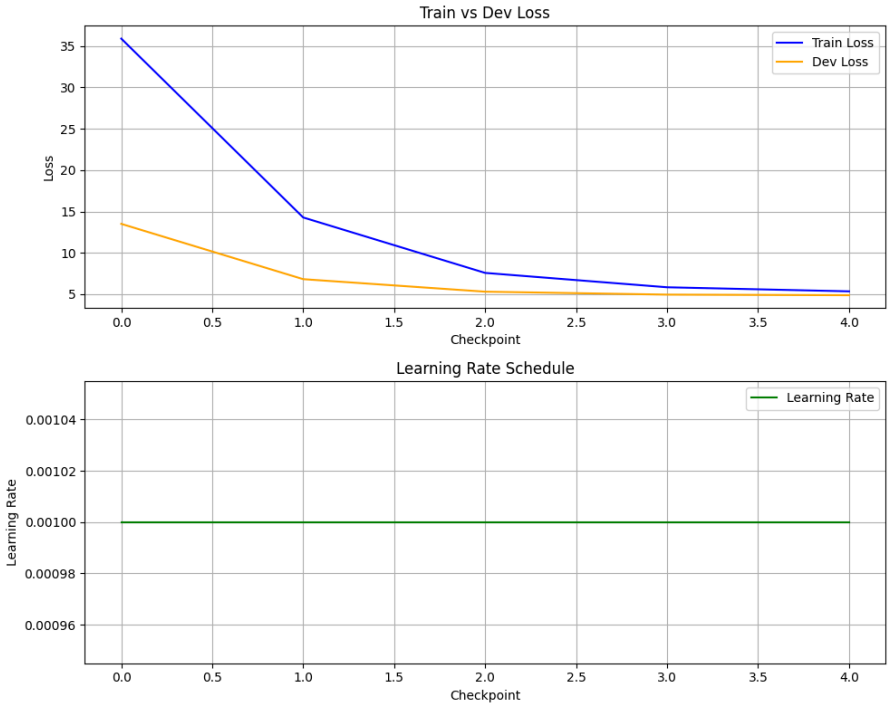
To make it easier to understand the tests will be using an epoch based approach. The checkpoint size will be equal to the length of the dataset / batch size (1 epoch of the data) and then will multiply the checkpoint size by n to get the desired amount of epochs.

We have also increased the batch size to be 32. This means that the epoch size will be 943. With the default learning rate of 0.001.

To make this step easier, two parameters have been implemented into the train function, this will automatically switch the training to epoch based training rather than the original steps. These parameters are epoch_based, automatically set to false by default, and epoch_amount, which will be set to 0 by default.

Training Loss

This is run over 1 epoch with SGD, the lambda scheduler and the qa_nll, with 200 step checkpoints.



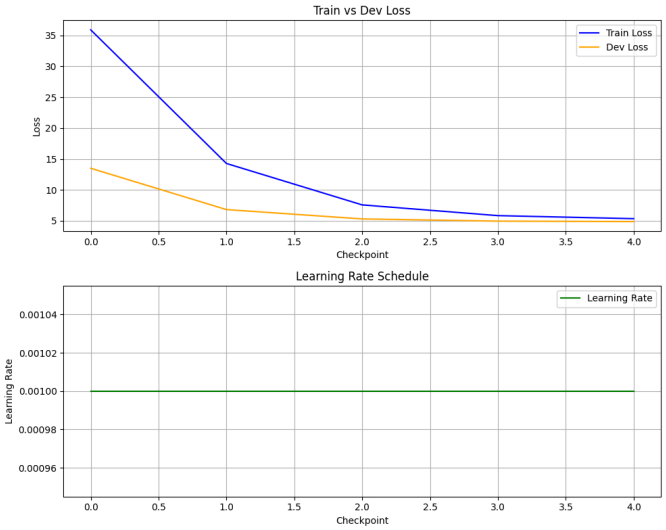
As we can see, as the checkpoints progress we see that both training and test loss decrease over time. In this case the learning rate is expected to be stable as we are using the lambda function.

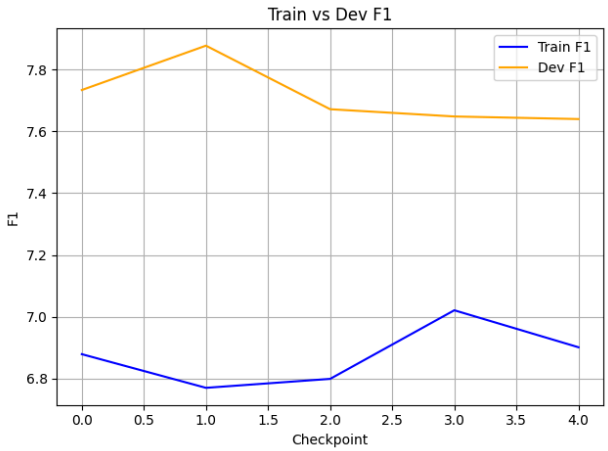
Extracted Metrics

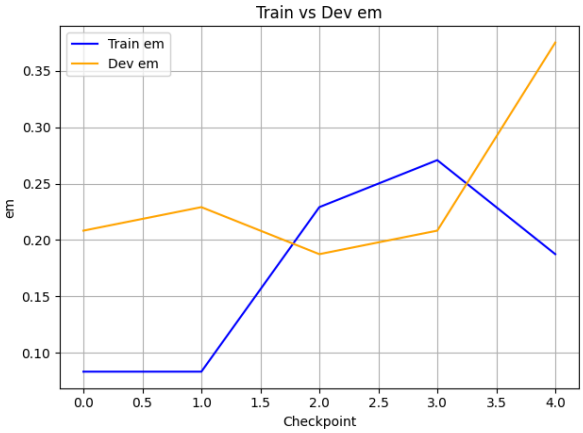
From this fixed case, a full evaluation cycle can be performed.

```
100%|          | 1309/1309 [01:29<00:00, 14.68it/s]
TEST  loss 4.960312  F1 7.806344  EM 0.353559
F1: 7.8063  |  EM: 0.3536  |  Loss: 4.960312
```

I have also implemented a function to plot tracked values.







Looking at the loss, f1 and em scores as they are logged, it shows that from the training and evaluation scaffolds are metrics can be extracted and used to make interpretations about training patterns. In this case the f1 is slightly fluctuating, but the em has an increasing trend over the course of one epoch but it is training as the loss is decreasing.

The model shows signs of convergence as the rate of loss decrease slows in later steps, indicating the training is stabilising, with a consistent slope downwards that is mirrored across both train and test data.

Experimental Investigation (Stage III)

Machine reading comprehension aims to extract parts of text that are most likely to contain the answer to a question by marking the start and end token in a sequence of tokens. QANet is designed to remove recurrence and rely on convolutions for local interactions between tokens, and self-attention for global interactions between tokens. The convolution portion is important for detecting short range compositional patterns in context and query representations.

Standard QANet uses depth wise separable convolutions, which have constant kernel size and therefore non-variable receptive fields per level. The fixed kernel size biases the model toward one scale of local feature extraction, which does not align with the nature of structure of written language. Inception style modules allow multiple receptive fields to be processed in parallel, which means subsequent steps will avoid local-scale bias. This could help QANet answer questions more effectively by capturing relationships that lie outside the normal receptive field of the constant-sized depth wise separable convolution. Slightly shorter or slightly longer semantic patterns can be recognized.

The Inception Model

An inception style convolution layer has parallel branches that take the same input, and run different sized convolutional filters on it. There is optionally a bottleneck projection to increase efficiency. The branches are either summed or concatenated together and outputted. This combines information from multiple feature scales. Small filters might catch short-range syntactical dependencies, where large filters might catch phrase-level dependencies. Doing these all in one step avoids excluding information in the output to the next step.

QAnet already contains local and global modeling by combining convolutions and self attention. This experiment aims to investigate whether standard QANet is bottlenecked by local feature encoding, or whether it is already sufficient.

Common Parameters

Within all three experiments there are common parameters that are used for all models and in training. Any alternative to these parameters are explained within each experimental section. Below is a table of the parameters, their value and a justification of the parameter.

Parameter	Value	Justification of value
Epochs	1	To save runtime, whilst also allowing for enough checkpoint steps to display training behaviour.
Batch Size	32	Large enough to be suitable to use of google collab GPUs, whilst allowing for some noise to generalise the model.
Learning Rate Scheduler	Cosine Annealing	Smoothly adjusts learning rate, without keeping it constant.
Optimiser	Adam	Theoretically converges quickly using the limited amount of epochs, also utilises momentum.
Loss	Qa_nll	Used as the experiment is depicting the spans of questions rather than classification.
All Others	Default	Used as the report is more interested in the training behaviour, the impact of multiple different parameters is for further investigation and they are kept constant to control the scope of the investigation.

All hypothesis experiments are conducted using the comparison.py file. It is run through assignment1.ipynb and all results are stored within _results. For all logs and graphs proceed to the _results section of the linked google drive.

Hypothesis 1

The inception based model will increase performance through F1 and EM metrics on the testing set as it is better at capturing multi-scale contextual interactions. The model will accelerate convergence resulting in a lower loss at each epoch relative to the baseline. The Inception based model will have an increased cost of computational power and parameter count when compared to the base model.

Implementation

The proposed modification is replacing every depth wise separable convolution module with a standard inception module that has 1d convolutions with kernel sizes of 1, 3, 5, and 7. The branch outputs will be concatenated together. This can be seen within the code base of inception.py. For ease of access the original codebase within train.py has been modified to allow for a parameter called use_inception, when set to True it will replace all depthwise modules within the encoder block. The training runs for a single epoch, with the final evaluation being done on the full SQuAD Dev set.

To accurately compare performance there are three sub hypothesis that are explored to see how well the affects work:

Hypothesis 1: Increase F1 and EM on the testing set by capturing multi-scale interactions
Hypothesis 2: Accelerate convergence resulting in faster reduction in training loss
Hypothesis 3: Increase compute cost (single epoch training time) and parameter count

Experimental Setup

To ensure experimental accuracy across research questions, the same initial base parameters were used to create a baseline model. This baseline will be used to compare the effects after modifying the implementation to answer the hypothesis and will remain the same across all variations.

To evaluate the effect of the architectural modification, performance should be measured using Exact Match and token-level F1 on the validation and test sets, since these are standard for extractive QA. To determine whether the change meaningfully improves optimization, training loss, validation F1 over time, and variance across random seeds should also be reported. Because the Inception module increases complexity, parameter count and training time per epoch are necessary for a fair comparison.

To ensure robustness the experiment is conducted across three different seeds and averaged out to get the performance over time.

Results

After running all three training and evaluation models, we get the following numerical results. Because SQuAD does not release the official test split, we evaluate on the Dev set, which serves as a proxy for the test set.

Metric	Baseline (mean +/- std)	Inception (mean +/- std)
Test F1	18.7008 +/- 0.4638	20.1654 +/- 0.1835
Test EM	10.3679 +/- 0.5696	11.2152 +/- 0.1728
Test Loss	3.5019 +/- 0.0094	3.3651 +/- 0.0230
Best Dev F1	19.2371 +/- 0.6255	20.9466 +/- 0.2268
Best Dev EM	10.9722 +/- 0.7574	11.9583 +/- 0.2211
Time / Ckpt (s)	91.1216 +/- 2.6212	125.0836 +/- 0.1548
Total Train Time (s)	455.6082 +/- 13.1060	625.4178 +/- 0.7740
Trainable Params	1,345,440	1,557,360
Total Params	17,257,440	17,469,360

When plotted the results can be more easily seen.

Experiment 1 Discussion:

Upon introducing an inception block, F1 and EM increased, and loss decreased with non-overlapping standard deviation ranges over the three random seed runs compared to the baseline. A possible explanation for this is that multiscale receptive fields at each layer capture dependencies at different ranges. Another explanation is the increased parameter count and model capacity. While these results suggest that inception modules increase performance, they could be improved upon with more random seeds and longer multi-epoch runs. In the subsequent hypotheses, we investigate how changes in addition to the inception module affect its performance.

Hypothesis 2

Inverting the encoder block to process global context (attention layers) before local context (inception based convolutional layers) will reduce F1 and EM scores and result in a higher final training loss compared to the baseline, as local feature extraction is more effective when applied to raw representations before global context is introduced.

The aim of this experiment is to see if context affects the way the ability to train. If the loss is lower and the f1 and em scores are higher when the architecture has the global and then local context is better, the model trains better when first generalising on the data before extracting the local context through the convolutional layers.

Implementation

To conduct this experiment, the encoder block function was copied into a new function within encoder.py, GlobalEncoderBlock, and the self attention and convolutional layers swapped. Within qanet.py if the inverse_encoder parameter is passed in as True, it will swap the encoder blocks so that the GlobalEncoderBlock will replace all of the original EncoderBlock function objects. As the results from Hypothesis 1 show that the inception model is superior to the original base convolutional model, this experiment was conducted using the inception model.

Experimental Setup

For this experiment a baseline model was set up using the base inception model with the common parameters. This base model uses the standard Encoder models within encoder.py that were modified within stages I and II. To compare against the baseline a secondary model was created using the GlobalEncoderBlock, this will be referred to as the experimental model. To fairly evaluate the effects of the change, all other parameters were kept constant.

As described in Hypothesis 1, within this experiment the models were trained across three different seeds and then averaged to get more robust results that are not dependent on the seed.

To investigate the hypothesis the experiment uses the standard loss, f1 and em scores. By comparing these values, it directly links back to the hypothesis but also shows that the difference in the setup is to do with the encoder change itself.

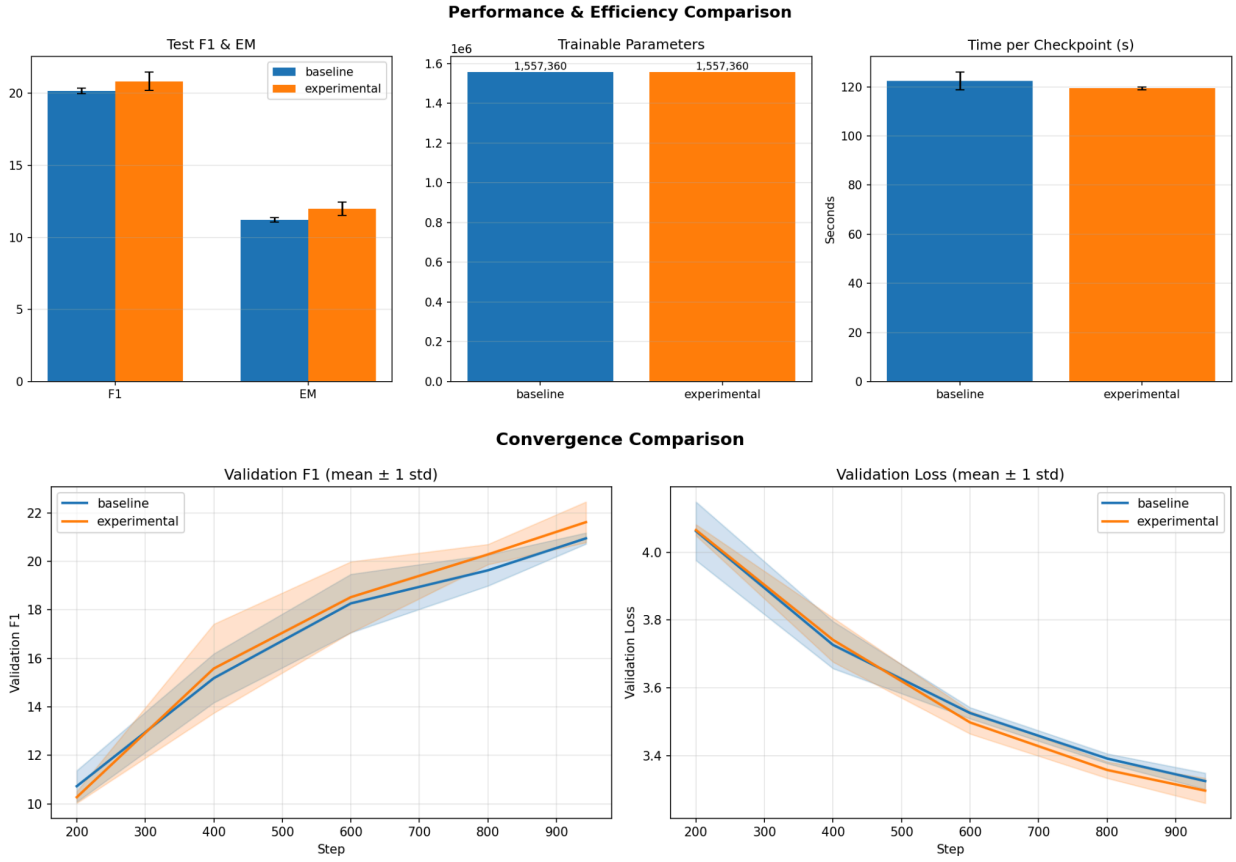
Results

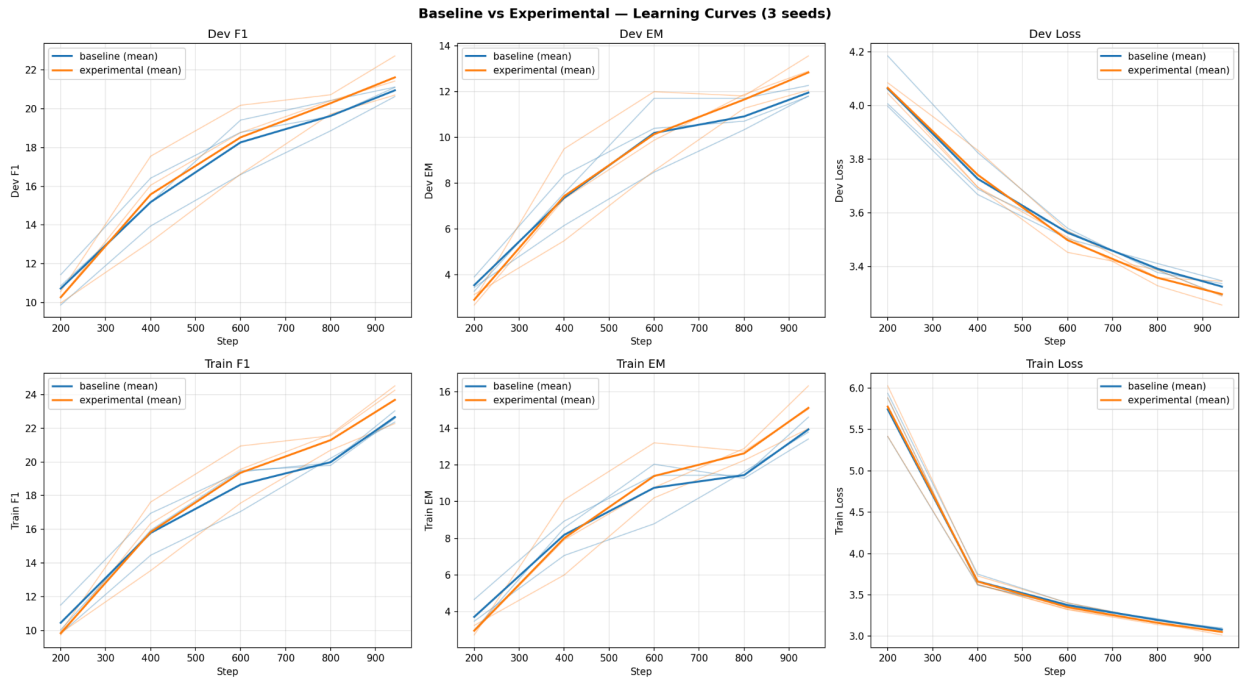
The three runs produced the following numerical data:

Metric	Baseline (mean +/- std)	Experimental (mean +/- std)
Test F1	20.1654 +/- 0.1835	20.8300 +/- 0.6454
Test EM	11.2152 +/- 0.1728	11.9987 +/- 0.4682

Test Loss	3.3651 +/- 0.0230	3.3365 +/- 0.0451
Best Dev F1	20.9466 +/- 0.2268	21.6157 +/- 0.8351
Best Dev EM	11.9583 +/- 0.2211	12.8403 +/- 0.6136
Time / Ckpt (s)	122.3997 +/- 3.6734	119.4287 +/- 0.5326
Total Train Time (s)	611.9987 +/- 18.3670	597.1437 +/- 2.6630
Trainable Params	1,557,360	1,557,360
Total Params	17,469,360	17,469,360

Whilst training this data was also logged, when plotted it can be shown as:





Experiment 2 Discussion:

Within experiment 2, the results show that there are no significant changes between switching the encoder block's local and global contexts. The test loss is within 0.03 of both models, with overlapping standard deviations, whilst the f1 and em scores are within 1 percentage point. However, it can be noted that the standard deviation of these values varies more in the experimental model. This could be because the global context makes the model more unstable, as applying attention first aggregates across all positions before any local feature extraction occurs, meaning the token level distinctions that the convolutional layers rely on may already be smoothed out by the time they are reached.

This similarity in results may be due to the overall architecture of qanet, with the introduction of residuals into each block, the effect of the attention layer is mitigated due to the residual containing all of the local context that would be lost and vice versa.Therefore there is very little effect in what order context processing occurs.

A future experiment to extend upon this reflection would be to explore the effect of the models based on answer length. The global model maybe be better for sentences that have longer rather than shorter relevant sections as the experimental model captures the global context first. A balanced mix of both longer and shorter answers could explain why both sections are close in results.

In terms of the hypothesis, it is refuted. As the f1 and em scores are slightly higher on average, but are mitigated by the wider standard deviation. This is also further supported in the test loss where there is little change when the encoder is swapped. Overall, as the performance between the two models is the same, the hypothesis that inverting the encoder block would reduce performance is not supported.

Hypothesis 3

Adding group normalisation to the Inception model will reduce training loss variance across steps and improve generalisation, compared to the same architecture using layer normalisation.

The aim of this experiment is to investigate the effect of regularisation on training stability and generalisation. If group normalization is better at generalising the model, the loss will be lower and will have higher f1 and em scores. The loss can also be used to show stability across steps.

Implementation

For this experiment a baseline model was set up using the base inception model with the common parameters. The baseline through parameters has its norm_name as layer_norm. This is changed within the second model with the norm_name being set to group_norm, with the group number being kept as the default parameter of norm_groups being 8. This implementation was already built into the original QANet implementation that was modified in stages I and II.

Experimental Setup

This experiment follows the same experimental setup as Hypothesis 1, utilising an inception module. However, to explore the hypothesis a baseline model is created that uses layer norm within the encoder block. To create a comparison model these will be changed to group norm for the second model, acting as our experimental model. All other parameters were kept the same to ensure that we are accurately comparing the changed normalisations.

To ensure robustness the experiment is conducted across three different seeds and averaged out to get the performance over time.

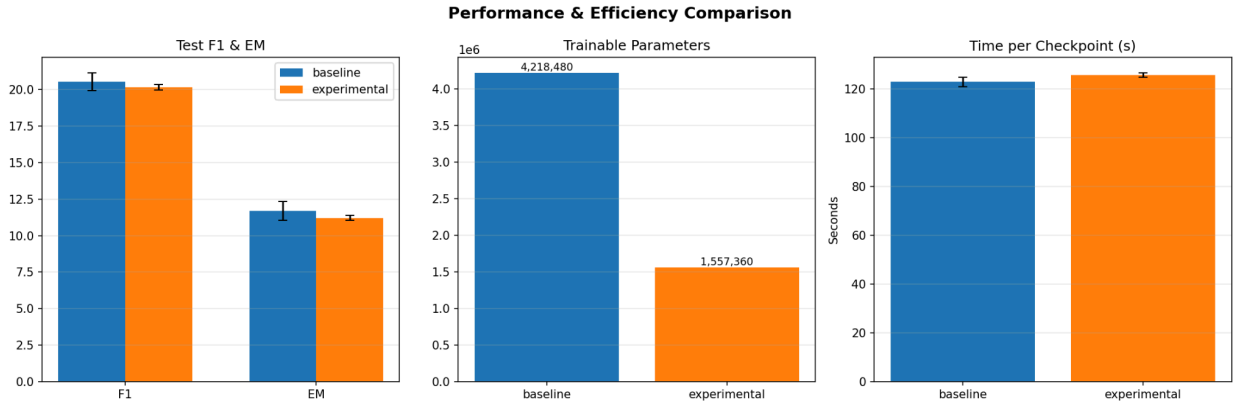
The metrics used to answer this hypothesis is f1, em score, as well as test and training validation loss curves.

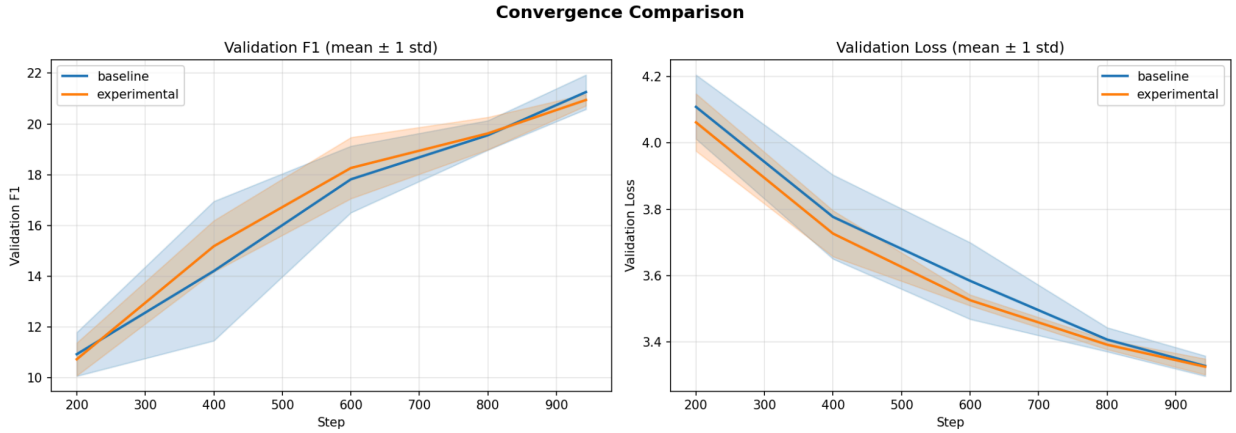
Results

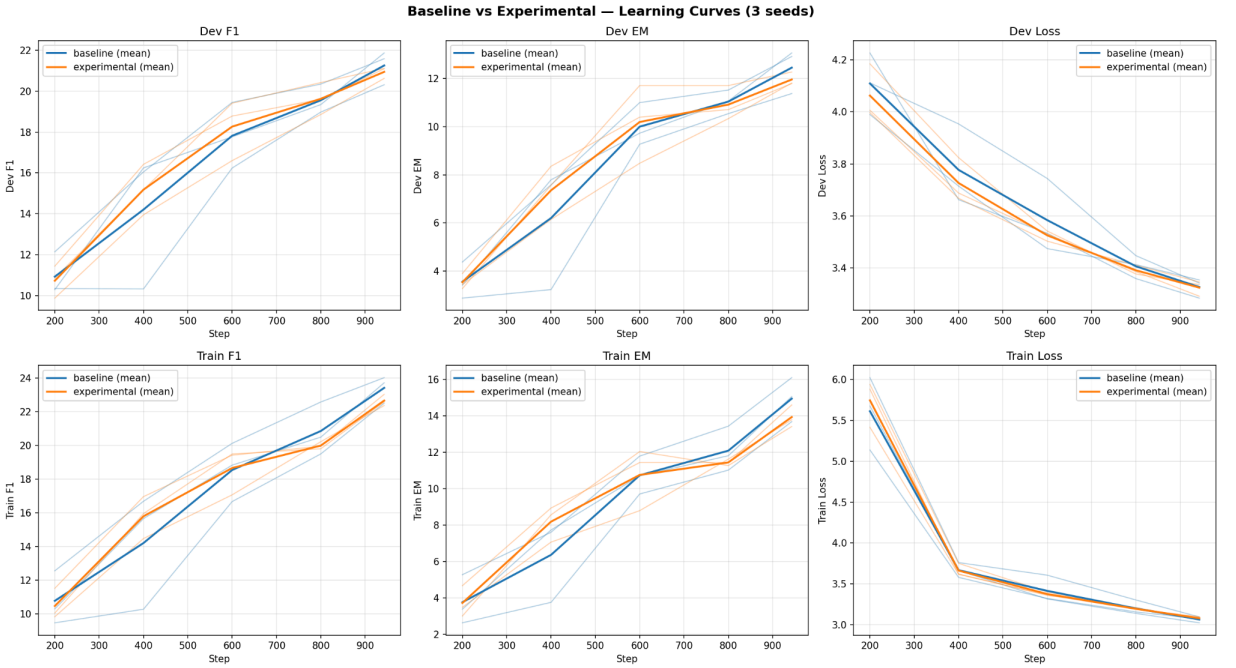
After running all three training and evaluation models, we get the following numerical results.

Metric	Baseline (mean +/- std)	Experimental (mean +/- std)
Test F1	20.5400 +/- 0.6126	20.1654 +/- 0.1835
Test EM	11.7057 +/- 0.6505	11.2152 +/- 0.1728
Test Loss	3.3737 +/- 0.0239	3.3651 +/- 0.0230
Best Dev F1	21.2581 +/- 0.6765	20.9466 +/- 0.2268
Best Dev EM	12.4514 +/- 0.7634	11.9583 +/- 0.2211
Time / Ckpt (s)	122.7760 +/- 1.9666	125.6343 +/- 0.8775
Total Train Time (s)	613.8799 +/- 9.8332	628.1716 +/- 4.3874
Trainable Params	4,218,480	1,557,360
Total Params	20,130,480	17,469,360

Whilst training this data was also logged, when plotted it can be shown as:


Performance & Efficiency Comparison
Test F1 & EM
baseline
experimental
F1
EM
Trainable Parameters
1e6
4,218,480
1,557,360
baseline
experimental
Time per Checkpoint (s)
Seconds
baseline
experimental


Convergence Comparison
Validation F1 (mean ± 1 std)
Validation F1
Step
baseline
experimental
Validation Loss (mean ± 1 std)
Validation Loss
Step
baseline
experimental


Baseline vs Experimental — Learning Curves (3 seeds)
Dev F1
Dev EM
Dev Loss
Train F1
Train EM
Train Loss
baseline (mean)
experimental (mean)
Step

Experiment 3 Discussion:

Within this experiment the results show that utilising group normalisation provides a slight advantage over layer normalisation across training, with the experimental data having a lower validation loss on average throughout the majority of the training, and although the average f1 and em scores are lower, their standard deviations are tighter, showcasing more reliable and consistent performance across epochs. This performance in validation loss may be due to the significant decrement in trainable parameters, as it acts as regularisation within the model and prevents overfitting. However the slight improvement is more likely to be due to the combination of the group norm with the inception convolutional layers, as the group normalisation allows for some local feature retention across channels, preserving the distinct feature representations produced by each parallel inception branch.

As all values for the experimental model have tighter standard deviations across steps, this model can also be considered more stable. This may be because layer norm operates over the entire feature map, causing it to be sensitive to input variation across different sequences.

In terms of the hypothesis this experiment partially supports the argument, as it provides a slightly better loss performance in a more consistent and reliable manner as indicated by the smaller standard deviation. However, the generalisation claim is only partially supported, as its f1 and em scores are marginally worse.

Limitations and Further Work of the Experiments

These experiments lack depth in terms of long term training, these experiments look at the initial training phases over a singular epoch where the training has the most significant impacts. This is designed as the experiments explore the training behaviour and general trends, not specifically the final accuracy of the models. However, it may be beneficial to explore the impacts of these experiments over multiple epochs in a stages approach, increasing the epochs until overfitting or a plateau in training is achieved. This will allow the overall impact of these experiments on the final accuracy and results of a fully trained model.

An additional method of comparison to get the results of hypothesis 2 and 3 would be to rerun the experiments using the base QANet implementation and compare against the current hypothesis 2 and 3. This would show if the results are generalized to the model or if they specifically only occur on an inception model.

Discussion

From both the debugging and experimental stages, it is evident that deep learning systems are very complex applications of multiple different interconnected components. Each component has a specific function that is combined in a specific way to reach a specific goal, in this case providing the relevant part of an input sentence to answer a given question. This modular nature introduces significant complexity as mistakes earlier in the training pipeline can completely influence the correctness and usefulness of the entire model.

This can be seen throughout the debugging process where there are common programming errors in implementation such as incorrect function calls or improper indexes when iterating over tensors. However, the majority of the pipeline issues are related to shape or dimension mismatches. This demonstrates how imperative it is to understand how inputs move throughout the network, exactly how they are transformed and how operations are needed on certain aspects of the pipeline. As if the incorrect dimensions or shape is being worked on, it will mean that the wrong weights are benign applied to the wrong data.

A similar pattern also emerged with investigating the mechanism issues, where a minor part of the corrected solutions occurred when an incorrect dimension is permuted or is iterated over. The only way to find these issues without going through each line of code would be to closely follow the shape of the tensors to conclude what went

wrong. However, the dominant class of mechanism issues were within calculations, where there is some small part of the equation within each component that was slightly incorrect. These often create difficult errors to diagnose by not producing errors but instead doing the intended effect to the incorrect scale, inverting the effect completely or skipping aspects of the architecture. During repeated components such as the encoder block or convolutional stages, these issues compound on each other, completely throwing out the training process, the accuracy of the model and the overall usefulness of the model. To prevent these mistakes within the training of deep learning models, it requires careful implementation of the correct formulas and equations and proper care with exactly how data is manipulated.

The goal of the QANet encoder is to efficiently transform text token sequences into a meaningful feature-space. It does so using convolutions, which have been thoroughly investigated in the Computer Vision field. One modification that was proved to work in GoogleNet is the Inception module. It was found that using multiple sizes of convolutions, models could better encode image data by combining information at multiple receptive fields. This report decided to experimentally investigate whether using Inception modules would similarly increase the performance of QANet. To do so, three hypotheses were formulated that investigated simple inception, attention first before inception, and normalisation within inception. The simple inception yielded better results than the baseline, likely due to the varying receptive fields and increased parameter count. When swapping context, so that it processed information globally first through self attention before convolutional inception layers there was no significant change, this may be due to the effect of residuals in the architecture making it robust to context dependant situations. Group normalisation within the inception module yielded more consistent results across random seed runs compared to LayerNorm, likely because LayerNorm is sensitive to input variation. Future work should explore longer training schedules and additional ablations to isolate the contributions of multi-scale structure versus parameter count, and to better understand how these modifications interact with attention mechanisms in QANet.

In summary, the nonfunctional pipeline was debugged, the broken training mechanisms were corrected to enable convergence, and experimental modifications were used to improve the QANet architecture. Systematically inspecting the codebase, running targeted tests, and resolving implementation errors strengthened our understanding of both the theory and practical behavior of deep learning systems. Building and evaluating our own modifications further developed this intuition, allowing us to apply insights gained from debugging to architectural design.

Across both stages, we observed that small errors and design changes propagate through interconnected modules to produce disproportionately large effects. This highlights the importance of careful implementation, rigorous validation, and thoughtful consideration of theoretical implications when developing and modifying deep learning models.

References

Alammar, J., & Grootendorst, M. (2024). Chapter 3: Looking inside transformer language models. In Hands-on large language models. O'Reilly Media.

GeeksforGeeks. (2025). Kaiming initialization in deep learning. https://www.geeksforgeeks.org/deep-learning/kaiming-initialization-in-deep-learning/

PyTorch Contributors. (n.d. -a). Adam - PyTorch 2.11 documentation. PyTorch. https://docs.PyTorch.org/docs/stable/generated/torch.optim.Adam.html

PyTorch Contributors. (n.d.-b). Learning rate schedulers - PyTorch main documentation. PyTorch. Retrieved April 13, 2026, from https://docs.PyTorch.org/cppdocs/api/optim/schedulers.html

PyTorch Contributors. (n.d -c). SGD - PyTorch 2.11 documentation. PyTorch. https://docs.PyTorch.org/docs/stable/generated/torch.optim.SGD.html

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. https://arxiv.org/abs/1706.03762
Xu, J., Sun, X., Zhang, Z., Zhao, G., & Lin, J. (2019). Understanding and improving layer normalization. arXiv. https://arxiv.org/abs/1911.07013

Yu, A. W., Dohan, D., Luong, M. T., Zhao, R., Chen, K., Norouzi, M., & Le, Q. V. (2018). QANet: Combining local convolution with global self-attention for reading comprehension. arXiv. https://arxiv.org/abs/1804.09541

GeeksforGeeks. (n.d.). Weight initialization techniques for deep neural networks. https://www.geeksforgeeks.org/machine-learning/weight-initialization-techniques-for-deep-neural-networks/

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (n.d.). Stochastic gradient descent. In Dive into deep learning. https://d2l.ai/chapter_optimization/sgd.html